

Write-Up Tugas Besar

Machine Learning from Scratch pada Kompetisi Kaggle

AI Lab Recruitment Task 2 (ai-lab-recruitment-task-2)

Nama	Kurt Mikhael Purba
NIM	13524065

1. TLDR

Tugas ini meminta saya mengimplementasikan Decision Tree Learning, Logistic Regression, dan SVM dari nol dengan NumPy untuk kompetisi klasifikasi tabular di Kaggle, lalu membandingkannya dengan versi scikit-learn dan melakukan submisi. Targetnya memprediksi `loan_status` dengan metrik macro F1. Prosesnya berjalan tiga tahap notebook yang saya petakan ke v1, v2, dan v3. Tahap pertama (v1) membangun pipeline robust dengan repeated stratified 5-fold cross-validation, tuning threshold exact untuk macro F1, dan feature expansion tanpa bocor ke test. Tahap kedua (v2) menambahkan CART cost-sensitive dengan bobot kelas positif di dalam Gini impurity. Tahap ketiga (v3) menguji eksperimen terkontrol memasukkan `person_id` sebagai fitur setelah audit train-only menunjukkan struktur source-order yang kuat. Submisi terbaik saya mencapai skor publik 0.88735 dengan satu CART dari scratch, diikuti 0.86571 (weighted CART), 0.85409 (CART robust), dan 0.84573 (baseline pertama).

2. Problem Overview

Kompetisinya adalah *AI Lab Recruitment Task 2*, klasifikasi biner tabular untuk kelayakan pinjaman. Setiap baris berisi karakteristik pemohon seperti `person_age`, `person_income`, `person_home_ownership`, `person_emp_length`, `loan_amnt`, `loan_int_rate`, `loan_percent_income`, `credit_score`, dan `previous_loan_defaults_on_file`. Targetnya `loan_status` bernilai 0 atau 1 dengan kelas positif sebagai minoritas. Kompetisi memakai macro F1, rata rata F1 kelas 0 dan kelas 1, jadi performa di kelas mayoritas saja tidak cukup.

Aturannya tegas: ketiga algoritma harus from scratch dan hanya boleh memakai library komputasi matematis seperti NumPy. Pandas hanya untuk membaca tabel, matplotlib untuk visualisasi, dan scikit-learn hanya sebagai pembanding sekaligus penyedia metrik. Tidak boleh ada ensemble method atau algoritma di luar yang disebutkan untuk submisi. File submisi harus berisi `person_id` dan `loan_status` dengan urutan identik terhadap test.

3. Decision Tree Learning

Saya memilih CART, bukan ID3 atau C4.5. Alasannya dikarenakan CART menangani fitur numerik kontinu secara alami lewat threshold biner, sedangkan ID3 butuh diskretisasi dan C4.5 menangani multiway split. Selain itu pembanding scikit-learn `DecisionTreeClassifier` juga berbasis CART, jadi perbandingannya fair.

Implementasi saya menumbuhkan pohon secara rekursif. Pada tiap node, semua kandidat split dicari dengan mengurutkan nilai fitur, lalu mengambil titik tengah di antara dua nilai yang berbeda. Kandidat yang membuat salah satu child lebih kecil dari `min_samples_leaf` dibuang. Split terbaik dipilih dengan memaksimalkan penurunan Gini impurity

$$G(S) = 1 - p_0^2 - p_1^2, \quad \Delta G = G(\text{parent}) - \frac{n_L}{n}G(S_L) - \frac{n_R}{n}G(S_R).$$

Pertumbuhan berhenti ketika kedalaman maksimal tercapai, jumlah sampel kurang dari `min_samples_split`, atau node homogen. Leaf menyimpan probabilitas kelas positif, dan feature importance diakumulasi dari gain dikali bobot sampel node. Pada versi awal saya membatasi jumlah kandidat threshold (48) agar cepat, tetapi di v1 saya ganti dengan exact search karena kualitas split naik meski waktu training bertambah.

Versi kedua menambahkan weighted Gini. Perhatikan bahwa Macro F1 menghukum kelas minoritas sehingga bobot kelas positif diikutkan dalam perhitungan impurity dan estimasi probabilitas leaf. Parameter `positive_class_weight` ini dituning dari train saja lewat OOF CV, bukan dari leaderboard. Ini tetap satu pohon tunggal, bukan ensemble, jadi tidak melanggar aturan. Perbandingan dengan `DecisionTreeClassifier` scikit-learn pada split validasi yang sama memberikan pola yang mirip, dan saya tidak menemukan anomali besar antara keduanya.

4. Logistic Regression

Logistic Regression dari scratch memakai sigmoid untuk menghasilkan probabilitas kelas positif,

$$\hat{p} = \sigma(Xw + b) = \frac{1}{1 + e^{-(Xw+b)}},$$

dengan objective weighted binary cross-entropy plus regularisasi L2. Kelas minoritas diberi bobot seimbang yang dihitung manual. Optimisasinya memakai mini-batch dengan optimizer Adam, yang nanti saya jelaskan lebih dalam di bagian bonus. Ada early stopping berbasis patience terhadap loss penuh per epoch, dan saya mencatat loss history untuk visualisasi proses training.

Di v1, model linear mendapat bantuan berupa nonlinear feature expansion yang target-independent: basis threshold dari quantile pada fitur kunci seperti `loan_percent_income`, `loan_int_rate`, dan `credit_score`, plus interaksi dengan `person_home_ownership`. Transformasi ini bukan model baru, prediktor akhirnya tetap Logistic Regression, dan cut-point quantile selalu dipelajari dari training fold saja.

5. Support Vector Machine

SVM saya buat sebagai linear SVM dengan label $+1/-1$ dan objective hinge loss plus regularisasi,

$$\frac{\lambda}{2} \|w\|^2 + \frac{1}{n} \sum_i \max(0, 1 - y_i(w^T x_i + b)).$$

Optimisasi memakai mini-batch subgradient descent dengan learning rate yang menurun seiring epoch. Gradien hanya menerima kontribusi dari observasi yang marginnya di bawah satu. Kelas yang tidak seimbang ditangani dengan sample weight, dan prediksi diambil dari tanda `decision_function`. Saya membandingkannya dengan `LinearSVC` scikit-learn pada preprocessing yang identik.

Perhatikan juga bahwa SVM saya linear sehingga performanya di data yang tidak linear kalah dibanding CART dan feature expansion yang sama seperti LR tidak cukup mengejar. Di akhir, SVM tidak dipakai untuk submisi utama, tapi tetap dipertahankan sebagai implementasi wajib dan pembandingan.

6. Validation Strategy

Strategi validasi naik bertahap. Baseline awal memakai stratified holdout 80/20 yang diimplementasikan sendiri di NumPy. Setelah itu saya beralih ke repeated stratified 5-fold cross-validation dengan prediksi out-of-fold (OOF). Preprocessing selalu di-fit ulang di dalam setiap fold agar tidak bocor, dan semua keputusan hyperparameter serta threshold hanya melihat train.csv.

Macro F1 sensitif terhadap threshold. Threshold default 0.5 untuk LR dan 0.0 untuk SVM tidak selalu optimal, apalagi dengan kelas tidak seimbang. Saya menulis fungsi exact threshold optimization yang menyortir skor OOF lalu menghitung macro F1 kumulatif dalam $O(n \log n)$, dan memilih threshold yang memaksimalkan macro F1. Dua atau tiga kandidat konfigurasi terbaik dari tuning seed tunggal lalu diuji ulang pada tiga seed CV (42, 2026, 777), dan pemenangnya dipilih dari rata rata repeated OOF macro F1 dengan threshold diambil dari median antar repeat.

Hasil lokal dan leaderboard publik cukup konsisten. Pada validasi OOF, varian ID-aware CART keluar sebagai yang terbaik, dan memang submisi `submission_best_scratch_v2_plus.csv` mendapat skor publik tertinggi. Perbandingan skor publik seluruh submisi saya ada di Tabel 1.

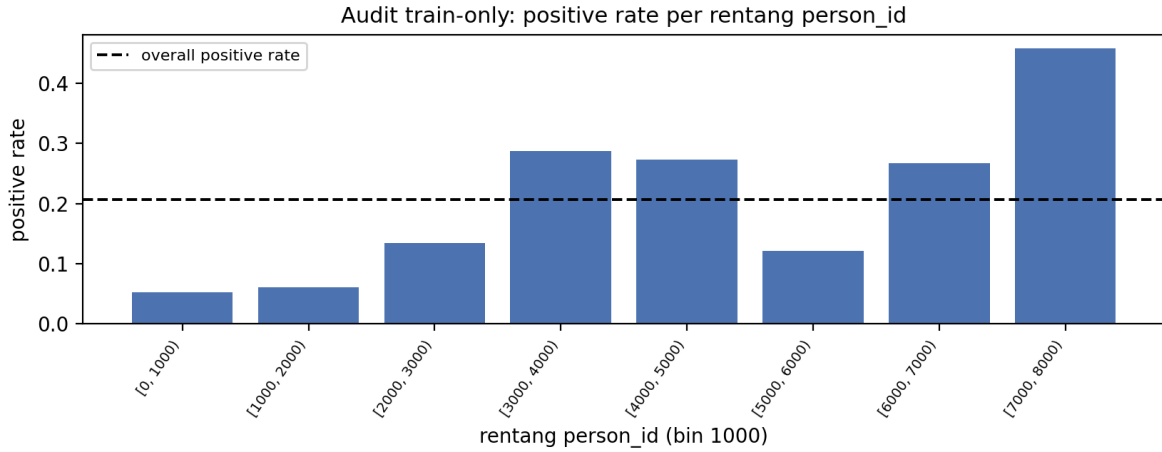
Submisi	Model	Skor Publik
<code>submission_best_scratch_v2_plus.csv</code>	CART scratch dengan <code>person_id</code>	0.88735
<code>submission_best_scratch_weighted_cart_v2.csv</code>	Weighted CART scratch	0.86571
<code>submission_best_scratch_robust.csv</code>	CART scratch robust no leakage	0.85409
<code>submission_cart_scratch.csv</code>	CART scratch baseline	0.84573

Tabel 1: Skor submisi publik di leaderboard

Satu hal yang jujur saya akui di write-up ini: audit train-only di v3 menunjukkan `person_id` membawa struktur urutan yang kuat terhadap target, seperti terlihat di Gambar 1. Binning ID dengan lebar tetap menampilkan positive rate yang melompat antar blok. Eksperimen ID-aware lahir dari situ, sebagai sensitivity analysis, dan terbukti paling tinggi baik di OOF maupun leaderboard. Peningkatan ini bukan bukti bahwa ID adalah faktor kredit yang sah, melainkan bahwa split dataset ini mengandung source-order signal. Tidak ada aturan manual seperti `if person_id > X then 1`; split terhadap ID murni dipelajari sendiri oleh CART.

7. Percobaan yang Gagal

Tidak semuanya langsung naik. Beberapa pendekatan yang saya coba gagal atau tidak sesuai harapan:



Gambar 1: Contoh audit train-only: positive rate melompat antar rentang person_id

1. **Threshold default dan class weight sekaligus.** Pada versi awal, LR dan SVM memakai balanced class weight lalu tetap di-threshold di 0.5 atau 0.0. Kompensasi kelasnya dobel dan hasilnya tidak optimal. Di versi robust class weight saya matikan dan biarkan threshold OOF yang menangani ketidakseimbangan, hasilnya lebih baik.
2. **Pembatasan 48 kandidat threshold di CART.** Cepat, tapi split yang ditemukan lebih kasar. Exact search menaikkan skor, jadi subsampling threshold saya buang.
3. **Pohon dalam tanpa batas.** max_depth=None tanpa aturan lain membuat pohon overfit. Regularisasi lewat min_samples_leaf (40) dan min_samples_split (80) yang membuatnya kembali sehat.
4. **Feature expansion untuk LR dan SVM.** Sudah menaikkan skor, tapi keduanya tetap di bawah CART. Pada akhirnya LR dan SVM tidak pernah menang untuk submisi, walaupun tetap wajib dikerjakan.
5. **Submisi pertama.** submission_cart_scratch.csv dari tahap baseline holdout mendapat 0.84573. Peningkatan berikutnya lahir dari pipeline robust, bukan dari mengubah model dasar.

8. Pengembangan Lebih Lanjut

Masih banyak yang bisa dicoba jika waktu tidak terbatas. Saya ingin mencoba SVM nonlinear dengan kernel yang ditulis sendiri di NumPy, misalnya kernel RBF, karena keluarga SVM memang membolehkannya. Selain itu probabilitas leaf CART bisa dikalibrasi dengan Platt scaling atau isotonic regression supaya skor lebih rapi sebelum di-threshold. Ruang tuning positive_class_weight dan min_samples_leaf bisa diperlebar dengan random search, dan jumlah quantile pada expansion bisa dinaikkan dengan teknik yang lebih hemat memori. Untuk persoalan person_id, eksperimen permutasi ID bisa mengukur seberapa besar sinyal yang benar benar datang dari urutan data, sehingga kita tahu batas generalisasi model ke data dengan ID teracak.

9. Bonus

9.1 Algoritma Optimasi Tambahan: Adam

Optimasi tambahan di luar materi kelas yang saya pakai adalah Adam (Kingma dan Ba, 2015), dipakai untuk melatih Logistic Regression dari scratch. Kelas ini biasanya baru mengajarkan gradient descent biasa, dan Adam membedakan diri dengan menjaga estimasi momen pertama dan kedua dari gradien,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

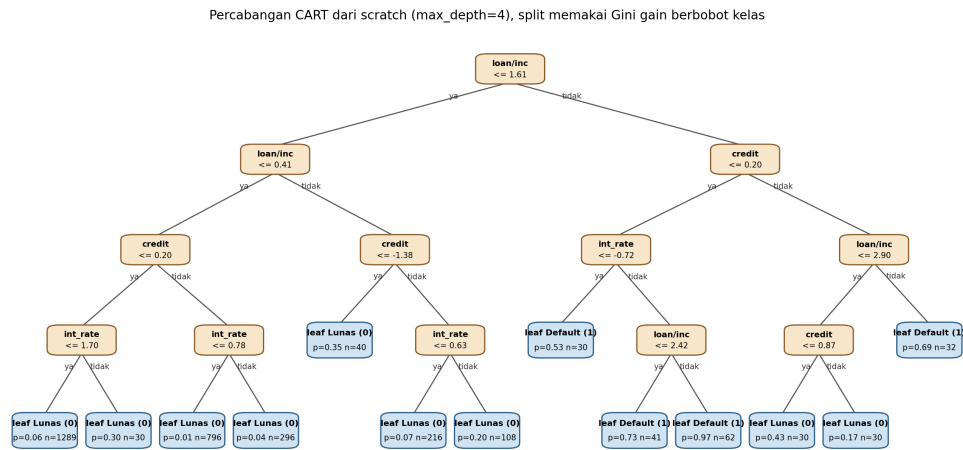
lalu mengoreksi bias karena inisialisasi nol dengan $\hat{m}_t = m_t / (1 - \beta_1^t)$ dan $\hat{v}_t = v_t / (1 - \beta_2^t)$, dan memperbarui parameter dengan

$$w \leftarrow w - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Alur kerjanya adalah m menjadi semacam momentum yang meredam osilasi, sedangkan v menormalkan langkah per koordinat sehingga koefisien yang gradiennya besar tidak melompat terlalu jauh dan koefisien yang gradiennya kecil tetap bisa bergerak. Hasilnya konvergensi lebih mulus dan tidak terlalu sensitif terhadap learning rate dibanding SGD polos, dan itulah yang saya amati dari loss history yang turun stabil tanpa bergetar.

9.2 Gambar Percabangan Tree

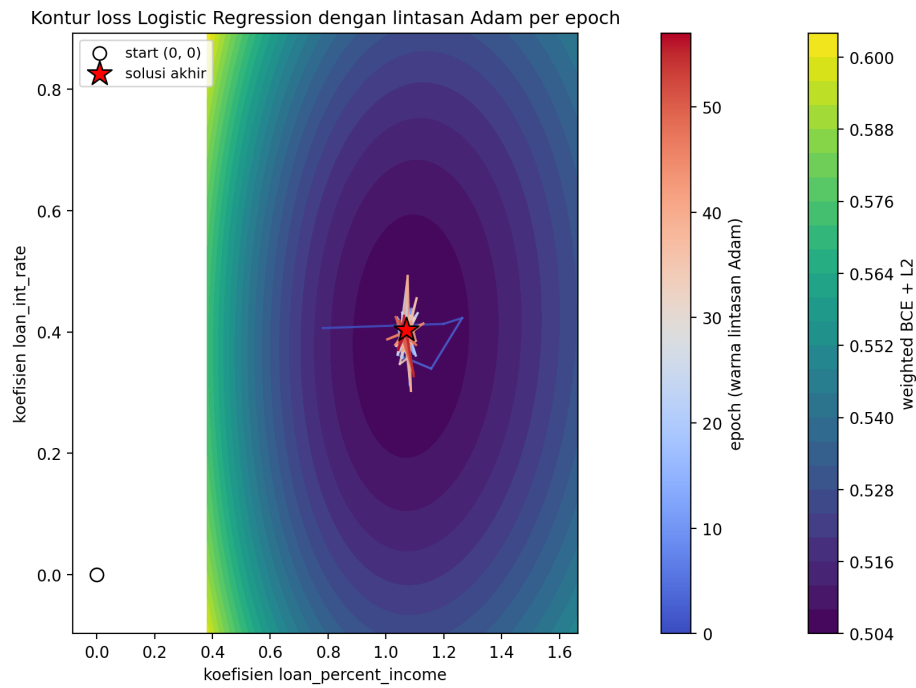
Gambar 2 adalah hasil rendering struktur `tree_` yang dihasilkan langsung oleh kelas `CARTClassifierScratch` dari v2. Karena data kompetisi tidak bisa dibawa keluar dari Kaggle, gambar ini dibuat dengan menjalankan ulang kelas yang sama pada data dengan skema yang identik agar tetap dapat direproduksi secara lokal. Tampak pohon memilih split pertama pada `loan_percent_income`, lalu bercabang ke `credit_score` dan `loan_int_rate`, konsisten dengan feature importance yang dihitung implementasi.



Gambar 2: Percabangan CART dari scratch, split memakai Gini gain, leaf berisi probabilitas positif

9.3 Visualisasi Proses Training Logistic Regression

Gambar 3 menampilkan kontur loss (weighted BCE plus L2) pada dua koefisien model yang dilatih dari nol, dengan lintasan parameter yang dilalui Adam per epoch diwarnai dari epoch awal ke akhir. Titik mulai ada di koordinat nol, lalu lintasan bergerak menuju dasar lembah loss yang diakhiri tanda bintang pada solusi akhir. Gambar ini dibuat dengan mencatat koordinat koefisien di setiap epoch selama training, tanpa mengubah kelas modelnya.



Gambar 3: Kontur loss Logistic Regression dan lintasan Adam selama training

10. Referensi

- [1] L. Breiman, J. Friedman, R. Olshen, dan C. Stone. *Classification and Regression Trees*. Wadsworth, 1984.
- [2] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [3] T. Hastie, R. Tibshirani, dan J. Friedman. *The Elements of Statistical Learning*, edisi 2. Springer, 2009.
- [4] C. Cortes dan V. Vapnik. Support-Vector Networks. *Machine Learning* 20(3):273-297, 1995.
- [5] D. P. Kingma dan J. Ba. Adam: A Method for Stochastic Optimization. *ICLR*, 2015.
- [6] F. Pedregosa dkk. Scikit-learn: Machine Learning in Python. *JMLR* 12:2825-2830, 2011.
- [7] Halaman kompetisi *AI Lab Recruitment Task 2*, <https://www.kaggle.com/competitions/ai-lab-recruitment-task-2>.